

Universidade Federal do Rio Grande do Norte
Departamento de Informática e Matemática Aplicada
Bacharelado em Tecnologia da Informação (BTI)

TelChain

Telemetria de Sensores IoT com Registro em Blockchain
Especificação da Aplicação Cliente/Servidor

Disciplina: DIM0438 — Redes de Computadores
Professor: Augusto José Venâncio Neto
Aluna: Yasmin Giordano Santos
Matrícula: 20240012578

Contents

1	Visão Geral do Projeto	3
1.1	Motivação	3
1.2	Repositório	3
2	Arquitetura da Aplicação	4
2.1	Visão Arquitetural	4
2.2	Componentes Principais	5
2.3	Concorrência	5
3	Estrutura da Blockchain	6
3.1	Transação (<i>Transaction</i>)	6
3.2	Bloco (<i>Block</i>)	6
3.3	Prova de Trabalho (PoW)	7
3.4	Raiz de Merkle	7
3.5	Selagem de Blocos	8
4	Protocolo TelChain	8
4.1	Características Gerais	8
4.2	Gramática das Mensagens	8
4.2.1	Autenticação	8
4.2.2	Mensagens de Sensor	9
4.2.3	Mensagens de Dashboard (consulta)	9
4.2.4	Mensagens de Dashboard (eventos push)	10
4.2.5	Mensagens Utilitárias	10
5	Diagramas de Sequência	10
5.1	Handshake e Autenticação	10
5.2	Comunicação entre Sensor e Servidor	11
5.3	Dashboard: Assinatura e Eventos Push	12
5.4	Dashboard: Consulta e Auditoria da Cadeia	12
6	Funcionalidades	12
6.1	Servidor (<i>TelChainServer</i>)	12
6.2	Cliente Sensor (<i>SensorClient</i>)	13
6.3	Cliente Dashboard (<i>DashboardClient</i>)	13
7	Guia de Reprodutibilidade (<i>How-To</i>)	14
7.1	Pré-requisitos	14
7.2	Compilação	14

7.3	Execução	14
7.4	Credenciais Pré-configuradas	15
7.5	Teste de Integridade	15
7.6	Execução em Hosts Distintos	16
8	Decisões de Projeto	16
9	Conclusão	17

1 Visão Geral do Projeto

A **TelChain** é uma aplicação cliente/servidor que combina telemetria ambiental com uma blockchain didática. Sensores IoT simulados enviam periodicamente leituras de temperatura, umidade e pressão atmosférica por meio de um protocolo proprietário baseado em sockets TCP. Enquanto isso, o servidor central, após autenticar cada dispositivo, agrupa as leituras em transações e as registra em blocos encadeados protegidos por Prova de Trabalho (PoW), assegurando a imutabilidade dos dados. Por fim, clientes do tipo *Dashboard* operam em modo interativo, podendo inspecionar a cadeia sob demanda e receber notificações *push* de novos blocos e transações em tempo real, além de terem a capacidade de auditar a integridade de toda a estrutura.

1.1 Motivação

A ideia para o projeto surgiu do interesse em explorar a integração entre blockchain e a Internet das Coisas, bem como do aprofundamento nos fundamentos de comunicação por sockets em Java. Mais especificamente, a escolha de um cenário de telemetria ambiental permite exercitar, em um único sistema, os conceitos de protocolo de rede proprietário, concorrência com múltiplos clientes e registro imutável de dados por meio de uma cadeia de blocos simplificada com prova de trabalho.

1.2 Repositório

O código-fonte do projeto está organizado conforme ilustra a Figura 1. Na raiz situa-se o `build.sh`, encarregado da compilação e empacotamento, enquanto o código Java fica em `src/telchain/`, dividido em três pacotes: `blockchain` com a lógica de integridade dos dados, `client` com as interfaces de sensor e *dashboard*, e `server` com a implementação do ponto central de comunicação.

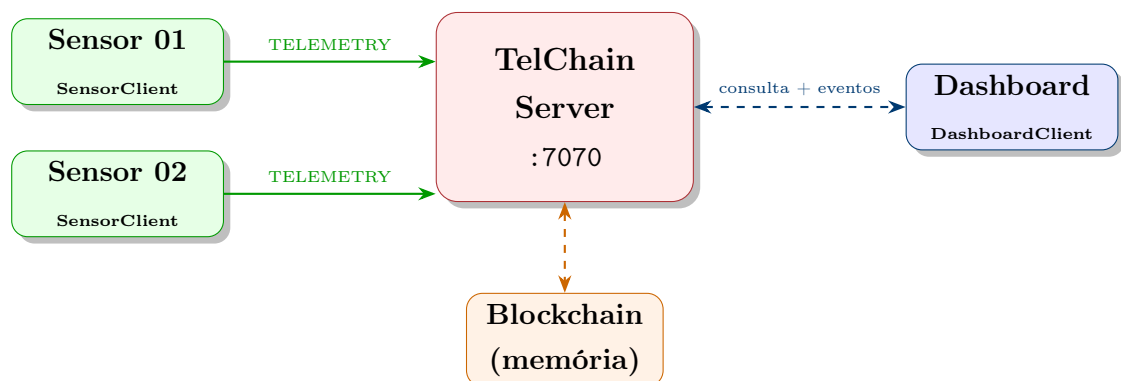


Figure 1: Estrutura de diretórios do projeto TelChain.

2 Arquitetura da Aplicação

2.1 Visão Arquitetural

A aplicação adota uma arquitetura estrela centralizada (*hub-and-spoke*): todos os clientes se conectam exclusivamente ao servidor, que atua como o *hub* central, enquanto sensores e *Dashboards* funcionam como *spokes*. Em outras palavras, não há comunicação direta entre clientes, toda mensagem sempre passa pelo *hub*. Esse aspecto, além de seguir a especificação dada para o projeto, simplifica o protocolo e elimina a necessidade de um mecanismo de consenso distribuído.

Figure 2: Arquitetura geral do TelChain – sensores publicam, servidor persiste na blockchain e *dashboards* monitoram via push/pull.

2.2 Componentes Principais

A composição do sistema pode ser resumida como um conjunto de partes que cooperam entre si para realizar a coleta, o registro imutável e o monitoramento dos dados. A organização desses componentes espelha a arquitetura cliente/servidor e a separação entre a lógica de rede, o núcleo da blockchain e as interfaces de usuário, conforme detalhado na Tabela 1.

Table 1: Componentes do sistema TelChain.

Componente	Responsabilidade
TelChainServer	Aceita conexões TCP na porta 7070; autentica clientes; delega o tratamento de cada conexão a um ClientHandler ; agenda a selagem periódica de blocos a cada 30 s.
ClientHandler	Executa em thread própria; parseia o protocolo TelChain; despacha comandos; controla inscrições de eventos por cliente.
Blockchain	Estrutura de dados encadeada e thread-safe; gerencia o pool de transações pendentes; sela blocos automaticamente ao atingir 5 transações ou por gatilho do timer.
Block	Representa um bloco imutável; calcula a raiz de Merkle das transações; realiza a Prova de Trabalho (prefixo "00" no hash SHA-256).
Transaction	Value object imutável; armazena leitura de telemetria; calcula o hash SHA-256 de seus campos.
SensorClient	Simula sensor IoT com random walk gaussiano; envia TELEMETRY periodicamente; faz ping a cada 10 envios.
DashboardClient	Interface interativa para auditoria; mantém thread leitora para receber eventos push; oferece comandos chain , verify , block N , stats , etc.

2.3 Concorrência

Para se proteger, o servidor adota o modelo *thread-per-client*, criando uma nova **Thread** e um **ClientHandler** dedicado para cada conexão aceita, o que mantém o loop de aceitação desimpedido. Já a consistência da **Blockchain** é garantida por métodos **synchronized** em todas as operações mutantes, além da adoção de **CopyOnWriteArrayList** pela lista de clientes ativos para que iterações de *broadcast* não sofram com modificações concorrentes. Por fim, a selagem periódica de blocos é confiada a um **ScheduledExecutorService** de

thread única, que dispara o fechamento da *pool* a cada 30 segundos sem interferir no tratamento de requisições.

3 Estrutura da Blockchain

3.1 Transação (*Transaction*)

A transação é a unidade atômica de dados que trafega do sensor até a blockchain. Ela representa uma leitura ambiental pontual e imutável, cuja integridade é garantida por um hash criptográfico calculado no momento da criação. Uma vez incluída em um bloco, a transação torna-se parte permanente do registro distribuído, podendo ser auditada a qualquer momento por um *Dashboard*.

Cada leitura de telemetria gera uma transação com os campos:

$$TX = \langle deviceId, temperature, humidity, pressure, timestamp, hash \rangle$$

Já o hash é calculado como:

$$hash = \text{SHA-256}(deviceId \parallel temp \parallel hum \parallel pres \parallel ts)$$

3.2 Bloco (*Block*)

O bloco é a unidade de agrupamento que sela as transações pendentes e as incorpora definitivamente à cadeia. Cada unidade desse componente lógico é ligada criptograficamente ao seu antecessor por meio do campo `previousHash`, formando uma sequência linear e temporalmente ordenada. Ainda por cima, para ser considerado válido, um bloco precisa satisfazer a Prova de Trabalho (discutida posteriormente) e ter sua raiz de Merkle consistente com as transações que transporta. Desse modo, para a construção de um bloco válido, é necessário um processo de mineração que ocorre no momento da criação do mesmo, garantindo que nenhum bloco permaneça em estado incompleto no sistema.

Table 2: Campos de um bloco TelChain.

Campo	Tipo	Descrição
index	int	Posição do bloco na cadeia (gênesis = 0).
timestamp	long	Época Unix em segundos.
previousHash	String	Hash SHA-256 do bloco anterior (64 hex).
merkleRoot	String	Raiz da árvore de Merkle das transações.
nonce	int	Valor encontrado pela Prova de Trabalho.
hash	String	SHA-256 de (index ts prevHash merkleRoot nonce).
transactions	List	Transações incluídas no bloco.

3.3 Prova de Trabalho (PoW)

Para assegurar a imutabilidade da cadeia, o TelChain emprega um mecanismo de consenso baseado em Prova de Trabalho (Proof of Work). Antes de ser anexado à cadeia, cada bloco precisa demonstrar um custo computacional mínimo, resolvendo um quebra-cabeça criptográfico: encontrar um nonce que faça o hash do bloco satisfazer um critério pré-definido. Essa exigência dificulta que um agente malicioso modifique blocos passados, pois qualquer alteração exigiria re-minerar todos os blocos subsequentes.

Para realizar esse trabalho, o algoritmo de mineração incrementa o **nonce** até que o hash SHA-256 do bloco inicie com "00" (1 byte zero, equivalente a dificuldade de $2^8 = 256$ tentativas em média). Embora didática, a dificuldade é suficiente para demonstrar o conceito sem penalizar o desempenho.

3.4 Raiz de Merkle

A integridade do conjunto de transações dentro de um bloco é garantida por uma árvore de Merkle. Essa estrutura organiza os hashes das transações em uma árvore binária, cuja raiz (*Merkle root*) funciona como uma impressão digital única do lote. Qualquer modificação em uma transação — por menor que seja — altera o hash correspondente e, por propagação, a raiz, permitindo detectar adulterações sem examinar todas as transações individualmente.

A raiz é calculada recursivamente sobre os hashes das transações, combinando pares (SHA-256(esq + dir)). Quando o número de nós é ímpar, o último é duplicado, já listas

vazias produzem `SHA-256("EMPTY")`.

3.5 Selagem de Blocos

A selagem é o processo que consolida as transações pendentes em um novo bloco e o anexa permanentemente à cadeia. Para evitar que leituras de telemetria permaneçam por tempo indefinido apenas na *pool* de espera, o servidor dispara automaticamente a criação de um bloco em duas situações complementares:

1. **Por volume:** ao atingir 5 transações pendentes no pool.
2. **Por tempo:** a cada 30 segundos, se houver ao menos 1 transação pendente.

4 Protocolo TelChain

4.1 Características Gerais

Protocolo TelChain — Resumo

- **Transporte:** TCP (porta padrão 7070).
- **Codificação:** texto puro, UTF-8.
- **Delimitador de mensagem:** `\n` (uma mensagem por linha).
- **Separador de campos:** `|` (pipe).
- **Autenticação:** obrigatória antes de qualquer comando operacional.
- **Papéis:** SENSOR (escrita) e DASHBOARD (leitura + push).

4.2 Gramática das Mensagens

A seguir, a especificação formal de cada mensagem do protocolo, agrupada em tabelas por direção e propósito.

4.2.1 Autenticação

Table 3: Mensagens de autenticação.

Direção	Mensagem	Formato / Descrição
C → S	HELLO	HELLO <role> <device_id> <secret> Inicia handshake; <code>role</code> ∈ {SENSOR, DASHBOARD}.
S → C	WELCOME	WELCOME <session_id> TelChainServer Autenticação bem-sucedida; retorna ID de sessão único.
S → C	REJECTED	REJECTED <motivo> Credenciais inválidas; conexão encerrada pelo servidor.

4.2.2 Mensagens de Sensor

Table 4: Mensagens originadas por clientes do tipo SENSOR.

Direção	Mensagem	Formato / Descrição
C → S	TELEMETRY	TELEMETRY <temp> <hum> <pres> <timestamp> Envia uma leitura ambiental (valores com 1 casa decimal).
S → C	ACK	ACK TELEMETRY <timestamp> Confirmação de recebimento da leitura.

4.2.3 Mensagens de Dashboard (consulta)

Table 5: Mensagens de consulta do Dashboard.

Direção	Mensagem	Formato / Descrição
C → S	GETCHAIN	GETCHAIN Solicita o dump completo da blockchain.
S → C	BLOCK	BLOCK <idx> <ts> <prevHash> <merkleRoot> <hash> <nonce> <txCount> Cabeçalho de bloco (repetido para cada bloco).
S → C	TX	TX <blockIdx> <txIdx> <devId> <temp> <hum> <pres> <ts> <hash> Transação dentro do bloco (repetida para cada TX).
S → C	ACK	ACK GETCHAIN <timestamp> Sinaliza o fim da transmissão da cadeia.
C → S	GETBLOCK	GETBLOCK <index> Solicita detalhes de um bloco específico.
S → C	BLOCK+TX	Mesmos formatos acima, apenas para o bloco solicitado.
S → C	ACK	ACK GETBLOCK <timestamp>
C → S	VERIFY	VERIFY Solicita verificação de integridade da cadeia.
S → C	CHAIN_OK	CHAIN_OK <blockCount> <tipHash> <timestamp> Cadeia íntegra.
S → C	CHAIN_ERR	CHAIN_ERR <blockIdx> <motivo> Falha de integridade detectada no bloco indicado.

4.2.4 Mensagens de Dashboard (eventos push)

Table 6: Mensagens de assinatura e eventos push.

Direção	Mensagem	Formato / Descrição
C → S	SUBSCRIBE	SUBSCRIBE <event_type> Inscreve o dashboard para receber eventos do tipo indicado (NEW_BLOCK ou NEW_TX).
S → C	ACK	ACK SUBSCRIBE <timestamp>
S → C	EVENT	EVENT NEW_BLOCK <idx> <hash> <txCount> Disparado ao minerar novo bloco.
S → C	EVENT	EVENT NEW_TX <devId> <temp> <hum> <pres> Disparado ao receber nova telemetria.

4.2.5 Mensagens Utilitárias

Table 7: Mensagens utilitárias (comuns a ambos os papéis).

Direção	Mensagem	Formato / Descrição
C → S	PING	PING <timestamp> <i>Keep-alive</i> ; enviado automaticamente pelo sensor a cada 10 leituras.
S → C	PONG	PONG <timestamp> Resposta ao ping.
C → S	BYE	BYE Encerramento limpo da sessão.
S → C	BYE	BYE Confirmação; servidor fecha a conexão.
S → C	ERROR	ERROR <código> <mensagem> Erros (ex.: 400 Bad Request, 403 Forbidden, 404 Not Found).

5 Diagramas de Sequência

5.1 Handshake e Autenticação

Os diagramas abaixo representam a troca de mensagens entre cliente e servidor.

Cliente	Dir.	Servidor
HELLO <role> <id> <secret>	→	
	←	WELCOME <sessId> TelChainServer
(comandos operacionais)	→	

Figure 3: Fluxo de handshake bem-sucedido.

Cliente	Dir.	Servidor
HELLO <role> <id> <wrong-secret>	→	
	←	REJECTED Invalid credentials

Figure 4: Fluxo de autenticação rejeitada.

5.2 Comunicação entre Sensor e Servidor

SensorClient	Dir.	TelChainServer
TELEMETRY <temp> <hum> <pres> <ts>	→	
		<i>/addTransaction Blockchain/</i> →
	←	ACK TELEMETRY <ts>
	←	EVENT NEW_TX ... (broadcast)
PING <ts>	→	
	←	PONG <ts>
BYE	→	
	←	BYE

Figure 5: Sequência de comunicação – telemetria (envio de leitura e ACK), *keep-alive* e encerramento.

5.3 Dashboard: Assinatura e Eventos Push

DashboardClient	Dir.	TelChainServer
SUBSCRIBE NEW_BLOCK	→	
	←	ACK SUBSCRIBE <ts>
SUBSCRIBE NEW_TX	→	
	←	ACK SUBSCRIBE <ts>
	←	EVENT NEW_TX ...
	←	EVENT NEW_BLOCK ...

Figure 6: Dashboard inscrito em eventos – recebe push a cada nova TX e novo bloco.

5.4 Dashboard: Consulta e Auditoria da Cadeia

DashboardClient	Dir.	TelChainServer
VERIFY	→	<i>[verify() → Blockchain]</i>
	←	CHAIN_OK ...
GETBLOCK <N>	→	<i>[getBlock(<N>) → Blockchain]</i>
	←	BLOCK <N> ...
	←	TX <N> ...
	←	ACK GETBLOCK <ts>
GETCHAIN	→	
	←	BLOCK 0 ..., BLOCK 1 ..., ...
	←	ACK GETCHAIN <ts>

Figure 7: Auditoria – verificacao de integridade, consulta de bloco e dump da cadeia.

6 Funcionalidades

6.1 Servidor (*TelChainServer*)

- **Gerenciamento de conexões:** aceita múltiplos clientes simultaneamente via *thread-per-client*.

- **Autenticação:** valida credenciais pré-configuradas e atribui papel (`SENSOR` ou `DASHBOARD`).
- **Ingestão de telemetria:** recebe `TELEMETRY` de sensores e insere transações na blockchain.
- **Selagem automática:** por volume (5 TXs) ou por tempo (30 s).
- **Broadcast de eventos:** notifica dashboards inscritos com `EVENT|NEW_BLOCK` e `EVENT|NEW_TX`.
- **Consultas:** responde a `GETCHAIN`, `GETBLOCK` e `VERIFY`.
- **Keep-alive:** responde a `PING` com `PONG`.
- **Desligamento limpo:** *shutdown hook* encerra o agendador ordenadamente.

6.2 Cliente Sensor (*SensorClient*)

- Simulação de sensor IoT com *random walk* gaussiano (temperatura $\pm 0,5$ °C, umidade $\pm 1\%$, pressão $\pm 0,3$ hPa).
- Envio periódico configurável via argumento `interval_ms`.
- Ping automático a cada 10 envios.
- Thread leitora separada para receber ACKs sem bloquear o envio.
- *Shutdown hook* que envia `BYE` antes de encerrar.

6.3 Cliente Dashboard (*DashboardClient*)

- Assinatura automática a `NEW_BLOCK`, `NEW_TX` e `ALERT` ao conectar.
- Recepção assíncrona de eventos push em thread separada.
- Interface interativa com os comandos listados na Tabela 8.
- Log de eventos em memória (últimos 20 exibidos com `log`).
- Estatísticas de sessão (`stats`): blocos vistos, TXs recebidas, eventos registrados.

Table 8: Comandos interativos do DashboardClient.

Comando	Efeito
chain	Dump completo da blockchain (todos os blocos e TXs).
verify	Verificação de integridade criptográfica da cadeia.
block <N>	Detalhes do bloco de índice N.
stats	Estatísticas da sessão local.
log	Exibe o log de eventos recebidos.
sub	Re-inscrição em todos os eventos.
ping	Envia PING e aguarda PONG.
help	Exibe ajuda.
quit	Encerra a sessão (envia BYE).

7 Guia de Reprodutibilidade (*How-To*)

7.1 Pré-requisitos

- **Java Development Kit (JDK) ≥ 17** (qualquer distribuição).
- **Bash** (para o script `build.sh`); em Windows, usar WSL2 ou Git Bash.
- Porta **7070/TCP** disponível no host do servidor.

Verificar versão do Java:

```
1 java -version
2 javac -version
```

Listing 1: Verificação do JDK.

7.2 Compilação

A partir da raiz do projeto:

```
1 chmod +x build.sh
2 ./build.sh
```

Listing 2: Compilação e geração dos JARs.

O script compila todos os fontes em `out/` e gera três JARs executáveis em `dist/`: `telchain-server.jar`, `telchain-sensor.jar` e `telchain-dashboard.jar`.

7.3 Execução

Abrir quatro terminais a partir da raiz do projeto:

```
1 java -jar telchain-server.jar
```

Listing 3: Terminal 1 — Servidor.

```
1 java -jar telchain-sensor.jar localhost 7070 sensor-01 s3cr3t 3000
```

Listing 4: Terminal 2 — Sensor 01 (intervalo de 3 s).

```
1 java -jar telchain-sensor.jar localhost 7070 sensor-02 s3cr3t2 4000
```

Listing 5: Terminal 3 — Sensor 02 (intervalo de 4 s).

```
1 java -jar telchain-dashboard.jar localhost 7070 dash-01 d4shp4ss
```

Listing 6: Terminal 4 — Dashboard.

A partir daqui, é possível interagir com o *Dashboard* no Terminal 4 usando os comandos anteriormente discutidos na Tabela 8.

7.4 Credenciais Pré-configuradas

Table 9: Credenciais disponíveis no servidor.

Device ID	Senha	Papel
sensor-01	s3cr3t	SENSOR
sensor-02	s3cr3t2	SENSOR
sensor-03	s3cr3t3	SENSOR
dash-01	d4shp4ss	DASHBOARD
dash-02	d4shp4ss2	DASHBOARD

7.5 Teste de Integridade

No terminal do Dashboard, aguardar ao menos um bloco ser selado (5 TXs ou 30 s) e então:

```
1 > verify
2 > chain
3 > block 1
4 > stats
```

Listing 7: Sessão de auditoria no Dashboard.

Atenção

A blockchain é mantida exclusivamente em memória. Ao reiniciar o servidor, a cadeia é reiniciada com um novo bloco gênese e todos os dados anteriores são perdidos.

7.6 Execução em Hosts Distintos

Para executar sensores ou dashboards em máquinas diferentes do servidor, substituir `localhost` pelo endereço IP do host servidor:

```
1 java -jar telchain-sensor.jar <SERVER-IP> 7070 sensor-01 s3cr3t
    5000
```

Listing 8: Conexão remota.

Garantir que a porta 7070/TCP esteja liberada no firewall do servidor.

8 Decisões de Projeto

1. **Protocolo orientado a texto com delimitador pipe:** favorece depuração com `telnet/netcat` e legibilidade do código, em detrimento de eficiência de banda, o que não é problema para um projeto didático.
2. **Thread-per-client:** mais simples de implementar e depurar do que NIO (Non-blocking I/O) assíncrono, além de ser adequado para o número reduzido de conexões esperado.
3. **Blockchain em memória:** elimina dependências externas (banco de dados, sistema de arquivos) e simplifica a execução. Mesmo assim, a persistência pode ser adicionada com serialização JSON/binária sem alteração do protocolo.
4. **Dificuldade PoW de 1 byte:** garante demonstração rápida (sub-segundo na maioria dos casos) sem sacrificar a semântica do algoritmo.
5. **Modelo push/pull híbrido:** sensores usam push puro (envio periódico), enquanto dashboards usam subscrição push para eventos em tempo real e pull sob demanda para auditoria histórica.
6. **CopyOnWriteArrayList para clientes:** segurança em leituras concorrentes durante broadcasts sem bloqueio explícito, em troca de custo de escrita (baixo, pois conexões são eventos raros).

9 Conclusão

O TelChain cumpre todos os requisitos da atividade (elaborar um protocolo de rede proprietário no modelo cliente/servidor) sobre uma base de blockchain com Prova de Trabalho e árvore de Merkle. Para além da conformidade, a convergência de temas diversos como telemetria ambiental, IoT e blockchain transformados em uma aplicação que funciona ilustra um grande valor criativo. O resultado é um projeto que serviu como laboratório prático para explorar desde os fundamentos de redes até os mecanismos de consenso criptográfico, entregando um sistema executável com um único script de build.